

Process Management

1. Program vs Process

- A **program** is a passive entity — just code sitting on disk (e.g., the compiled binary `./f1` after `cc f1.c -o f1`). The source file lives in secondary storage; nothing is "running."
- A **process** is an active entity — a program that has been given an address space and is either executing or scheduled to execute. The moment a program is loaded into memory and gets address space, it becomes a process.

Interview soundbite: "A program is code at rest on disk; a process is that code in motion — loaded into memory, with its own address space, registers, and execution state."

Foreground vs background processes

- **Foreground process** — requires user interaction (e.g., `./a.out` run directly).
- **Background process** — runs without user interaction (e.g., `./a.out &`).

2. PID and the Process Tree

- **PID (Process ID)** — a unique identifier generated by the OS for every process.
- **PID 1** — the **init** process (also called the root process). It's the first process created when the OS boots and has no parent; every orphaned process eventually gets re-parented to it.
- **PID 0** — sometimes called the "lazy" process / scheduler/swapper process depending on the OS — a kernel-level bookkeeping entity, not a normal user process.

```
graph TD
    init["(1) init"] --> shell["(5001) shell ..."]
    shell --> main["(5003) main program"]
    main --> child["(5004) child process"]
```

Why processes have children: to achieve parallelism — a parent can hand off independent sub-tasks to child processes that execute concurrently.

Orphan process

A process whose **parent has terminated** before it. It gets adopted by **init** (PID 1), which eventually reaps and kills it once it finishes.

Zombie process

A process that has **completed execution**, but its parent hasn't yet called `wait()` to read its exit status — so the OS keeps a minimal trace (its PCB / exit code) around. It "neither lives nor dies": it's done running, but its entry lingers in the process table until the parent collects it.

Key distinction: Orphan = parent died first, child still running. Zombie = child died first, parent hasn't yet acknowledged it.

Rules to remember:

- A child cannot kill itself directly out of that relationship rule — it terminates and waits to be reaped.
- A child cannot kill its parent.
- A parent *can* kill its child.

3. Process Control Block (PCB)

Every process has an OS-maintained data structure — the PCB — holding everything the OS needs to manage it. When a process is killed, its PCB is cleared.

Contents of a PCB:

1. PID
2. CPU Registers (saved values, so execution can resume exactly where it left off)
3. Parent's PID
4. Memory management info (page tables, base/limit registers, etc.)
5. Process state (**most important field** — drives all scheduling decisions)
6. Owner/user of the process
7. Time and date of creation
8. Scheduling-related information
9. Priority of the process

The PCB is a **kernel data structure** and can only be accessed in kernel mode — user processes never touch it directly.

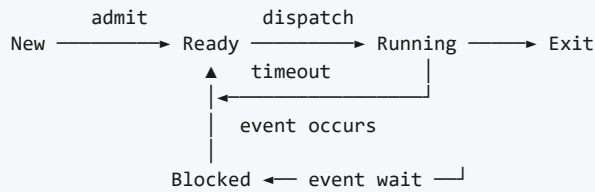
Process Table

The **process table** holds all PCBs as its entries — one row per active process. It exists at both:

- **User level** — a partial, filtered view (e.g., what `ps` shows you).
 - **Kernel level** — the full, authoritative table the OS actually uses for scheduling.
-

4. Process State Models

5-state model (the common one)



- **New** — process is being created; not yet admitted.
- **Ready** — has acquired all resources it needs and is waiting only for the CPU. Sits in the **ready queue** (0 to n processes).
- **Running** — actually executing on the CPU. In a single-processor system, exactly 0 or 1 process is in this state.
- **Blocked (waiting)** — waiting on an event (typically I/O) to complete. Sits in the **blocked/device queue** (0 to n processes).
- **Exit (terminated)** — finished; resources released.

Note: all these queues are conceptually the same *kind* of structure (a queue), just serving different purposes.

Reasons for process creation:

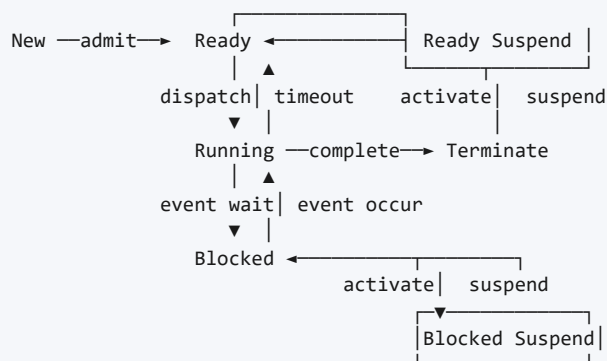
- Program execution (a user runs a new program)
- Created by a user/parent process (spawning a child)
- Interrupt-driven creation

Reasons for process termination:

- Normal completion, no error
- Killed by its parent
- Killed by some other process/the OS

7-state model

Adds two **suspended** states that move a process **out of main memory and into secondary storage**, freeing RAM under memory pressure:



- **Ready Suspend / Blocked Suspend** — the process still logically exists in that state, but its memory image has been swapped out to secondary storage (disk) rather than sitting in RAM.
- A process can move directly between **Blocked Suspend** and **Ready Suspend** if the event it was waiting on completes while it's still swapped out.

Swapping is expensive precisely *because* it involves secondary storage — this is why the OS only resorts to suspension under real memory pressure.

5. CPU-Bound vs I/O-Bound Processes, and Degree of Multiprogramming

Every process spends its life alternating between bursts of CPU computation and bursts of waiting on I/O. Classifying a process by which one *dominates* matters a lot for scheduling and memory decisions.

- **CPU-bound process** — spends most of its time doing computation, with relatively little I/O. Think: a video encoder, a number-crunching simulation, a compiler doing heavy optimization passes.
- **I/O-bound process** — spends most of its time waiting on I/O (disk reads, network calls, user input), with short bursts of CPU work in between. Think: a text editor waiting on keystrokes, a web server mostly waiting on network sockets.

"Bound" = where the time actually goes. A CPU-bound process is "CPU bound" because the CPU is the bottleneck resource it's waiting on; an I/O-bound process is "I/O bound" because I/O completion is what it's actually waiting on most of the time.

Why this classification matters

If a system had **only** CPU-bound processes, the ready queue would stay full (everyone wants the CPU) but the blocked/device queue would stay empty (nobody's waiting on I/O) — CPU utilization would look good on paper, but I/O devices would sit idle. The reverse is true if the system had **only** I/O-bound processes: the CPU would sit idle most of the time waiting for I/O to complete, even though it's the scarcest, most expensive resource.

Degree of Multiprogramming

Degree of multiprogramming = the number of processes kept resident in main memory at one time, available to be executed.

- Increasing the degree of multiprogramming means more processes are ready and waiting in memory, so the CPU is less likely to ever go idle (there's almost always *something* ready to run when the current process blocks on I/O).
- But this is only beneficial if the mix of processes is right — see below.

Why proper mixing of CPU-bound and I/O-bound processes matters

This is the actual **job of the long-term scheduler** (see the upcoming CPU Scheduling topic): to admit new processes into memory such that there's a healthy **mix** of CPU-bound and I/O-bound processes.

- **Why:** while a CPU-bound process is running on the CPU, I/O-bound processes in the system can be quietly doing their I/O in the background (keeping devices busy); and while an I/O-bound process is blocked waiting on I/O, a CPU-bound (or another ready) process can use the CPU instead of letting it idle.
- **The goal:** maximize the degree of multiprogramming *and* keep both the CPU and I/O devices busy simultaneously — which is only possible with a good mix, not just a high raw *count* of processes.

Interview soundbite: "A CPU-bound process's bottleneck is compute; an I/O-bound process's bottleneck is waiting on I/O. The long-term scheduler tries to admit a healthy mix of both into memory — too many CPU-bound processes leave I/O devices idle, too many I/O-bound processes leave the CPU idle. Getting this mix right is what actually lets you safely increase the degree of multiprogramming without wasting either resource."

6. Process Creation: `fork()`

```
#include <stdio.h>
#include <unistd.h>
int main() {
    fork();
    printf("ID: %d, Parent ID: %d\n", getpid(), getppid());
}
```

- `fork()` creates a **child process** as a near-exact copy of the calling (parent) process.
- Both parent and child continue executing **from the point right after `fork()`** — the code following it runs *twice* (once in each process).
- **Return value of `fork()`** is how each process tells itself apart:
- Returns `0` in the child.
- Returns the child's PID in the parent.
- It's *not* guaranteed which one runs first — the OS scheduler decides.

Example output:

```
ID: 1234, Parent ID: 1000    ← parent's line
ID: 1345, Parent ID: 1234    ← child's line
```

- **n calls to `fork()`** create **2^n** total process instances (each fork doubles the count, since every existing process — parent and every prior child — forks again).
- **`wait(NULL)`** inside the parent blocks it until the child finishes, guaranteeing the child runs to completion (or at least starts and is scheduled) before the parent proceeds.
- If the child calls **`sleep(n)`** and the parent finishes execution in the meantime, the child becomes an **orphan** — it's adopted by **`init`**, which reaps it once it eventually completes.

You can inspect a live process's PCB-derived info via the virtual `/proc/<pid>/` directory on Linux.

7. Context Switching

Context switch — saving the CPU state (registers, program counter, etc.) of the currently running process into its PCB, and loading the saved state of the next process to run, so execution resumes exactly where that process left off.

Mode switch vs. context switch:

- **Mode switch** — just flipping between user mode and kernel mode (e.g., during a system call), without necessarily changing *which process* is logically executing.
- **Context switch** — actually switching *which process* the CPU is running, which necessarily also involves at least one mode switch (into the kernel, to perform the switch, then back out).

Context switch time = mode switch time + additional overhead (saving/restoring registers, updating the PCB, memory management bookkeeping, cache/TLB effects).

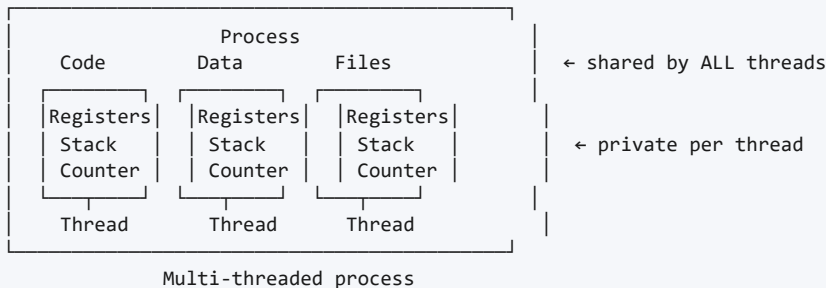
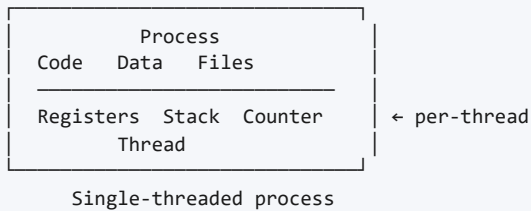
Context switches are pure overhead from the CPU's perspective — no useful work gets done during one. This is exactly why scheduling algorithms care about *minimizing the number of context switches*, not just picking a "fair" order.

8. Threads

Why threads exist

The whole point of a child process is to run tasks in parallel. But **creating a process is expensive** — the OS must allocate a fresh address space, page tables, and copy of the parent's resources for every single one. If all you actually need is "more than one thing running for the same task," a full process is overkill.

A **thread** is a lighter-weight unit of execution *within* a process.



- **Shared across all threads of a process:** code, data (global variables), open files, address space.
- **Private per thread:** registers, stack, program counter.

Why threading is faster than process creation: since threads share the process's existing address space, code, and open resources, creating a new thread just means allocating a new stack + register set — no new address space, no page table setup, no copying. This makes thread creation roughly **~100x faster** than process creation.

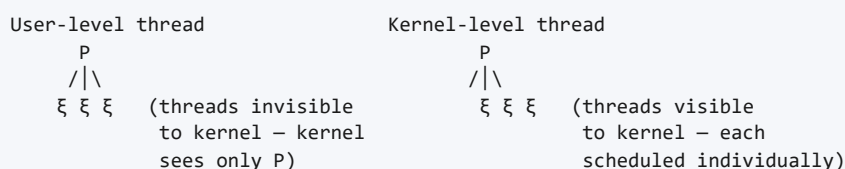
Child process vs. thread:

- A **child process** (via `fork()`) is a separate address space — it can even run a completely different program (`exec()`).
- A **thread** is only ever a *part* of its parent process — it always shares that process's code and data; it cannot exist independently.

9. User-Level vs Kernel-Level Threads

The critical question: **does the kernel know a given thread exists as a separate schedulable entity, or not?**

- **User-level thread** — the kernel has no idea it exists. The kernel only sees *one* process. Thread creation, scheduling, and switching are handled entirely by a user-space thread library.
- **Kernel-level thread** — the kernel is aware of each thread individually and schedules them itself.



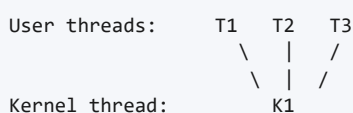
	User-level	Kernel-level
Who schedules it	The thread library, in user space	The kernel
Control	Full control over scheduling policy, in your own code	Kernel decides
Creation speed	Very fast (no kernel involvement)	Slower (kernel call needed)
Blocking problem	If one thread makes a blocking system call, the kernel blocks the entire process — since the kernel doesn't know other threads exist, it can't run them instead	Only the blocking thread is blocked; the kernel can freely schedule the process's other threads
Portability	Works without kernel support, portable across OSes	Depends on OS-level thread support

This blocking problem is the single biggest weakness of pure user-level threads, and it's exactly what motivates the hybrid model below.

10. Threading Models (Kernel ↔ User Thread Mapping)

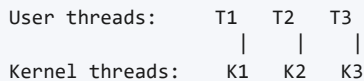
This is the piece that's genuinely new: **how many user-level threads map onto how many kernel-level threads?** Every real thread implementation is a choice along this axis.

10.1 Many-to-One (N : 1)



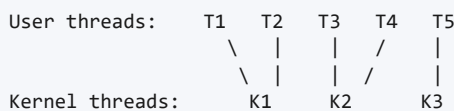
- Many user-level threads map onto a **single** kernel thread.
- Thread management (creation, scheduling, switching) happens entirely in user space, via a thread library — the kernel is unaware any of this is happening.
- **Advantage:** thread operations are extremely fast and efficient (no kernel involvement, no mode switches for scheduling).
- **Disadvantage:** exactly the blocking problem above — one thread's blocking system call blocks *all* threads of the process, since the kernel only sees one schedulable entity. Also, no true parallelism is possible even on a multi-core machine (only one kernel thread exists, so only one core can ever be used).
- **Example:** early green threads (original Java threads pre-JDK 1.2), GNU Portable Threads.

10.2 One-to-One (1 : 1)



- Every user-level thread maps to its **own** dedicated kernel thread.
- **Advantage:** solves the blocking problem completely — if one thread blocks, the kernel simply schedules another of the process's kernel threads. Also enables **true parallelism** across multiple cores, since each thread is independently schedulable by the kernel.
- **Disadvantage:** creating a user thread means creating a kernel thread too — more overhead per thread than the many-to-one model, and typically a practical limit on the number of threads a process can spawn (kernel resources aren't free).
- **Example:** this is what most modern general-purpose OSes actually use — **Linux (NPTL — Native POSIX Thread Library)** and **Windows** both implement 1:1 threading.

10.3 Many-to-Many (M : N)



- Many user-level threads are multiplexed onto a smaller-or-equal number of kernel threads — the library and kernel cooperate to schedule user threads onto whichever kernel threads are available.
- **Advantage:** combines the best of both — you can create as many cheap user-level threads as you like (fast creation like N:1), while still getting true parallelism and avoiding the total-blocking problem (because there are multiple kernel threads to fall back on, like 1:1).
- **Disadvantage:** significantly more complex to implement correctly — both the user-space library and the kernel need coordinated scheduling logic.
- **Example:** older Solaris threading (pre-Solaris 9) implemented this.

10.4 Two-level model (a refinement of M:N)

- Similar to Many-to-Many, but additionally allows a specific user thread to be **bound** directly to its own dedicated kernel thread (effectively a 1:1 mapping for just that thread), while other threads remain multiplexed M:N.
- Gives developers the flexibility to guarantee true concurrency for specific critical threads, while keeping the rest lightweight.

Quick comparison

Model	Parallelism on multi-core?	One thread blocks all?	Creation cost	Real-world use
Many-to-One	✗ No	✓ Yes (problem)	Very cheap	Early Java green threads
One-to-One	✓ Yes	✗ No	Higher (1 kernel thread per user thread)	Linux (NPTL), Windows
Many-to-Many	✓ Yes	✗ No	Cheap-ish, tunable	Older Solaris
Two-level	✓ Yes	✗ No	Tunable, plus dedicated binding option	Refinement of M:N

Interview soundbite: "Threading models describe how user-level threads map to kernel-level threads. Many-to-one is fast but blocks entirely on any blocking call and can't use multiple cores. One-to-one (what Linux and Windows actually use) fixes both problems at the cost of per-thread kernel overhead. Many-to-many tries to get the best of both by multiplexing many user threads over fewer kernel threads, at the cost of implementation complexity."

Interview Questions With Answers

Q1. What's the difference between a program and a process?

Answer: A program is passive code sitting on disk — just bytes, not executing. A process is that program loaded into memory with its own address space, registers, and execution state — an active, schedulable entity. The same program can be turned into multiple independent processes by running it more than once.

Q2. What is a zombie process, and how does it differ from an orphan process?

Answer: A zombie is a process that has *finished* executing, but its parent hasn't yet called `wait()` to collect its exit status — so its PCB/exit code lingers in the process table. An orphan is the opposite scenario: the *parent* has terminated while the *child* is still running; the child gets re-parented to `init` (PID 1), which will eventually reap it when it finishes.

Key distinction: Zombie = child dead, waiting on parent. Orphan = parent dead, child still alive.

Q3. What information does a PCB contain, and why must the "process state" field be there?

Answer: PID, CPU registers, parent PID, memory management info, process state, owner, creation time, scheduling info, and priority. The process state field is essential because it's the single value the scheduler checks to decide what to do with a process next — without it, the OS wouldn't know whether a process is ready to run, currently running, or blocked waiting on something.

Q4. Walk through the 5-state process model. What triggers each transition?

Answer: New → Ready happens via **admit**, once the process has all the resources it needs. Ready → Running happens via **dispatch** (the scheduler picks it). Running → Ready happens on a **timeout/preemption** (its time slice expires or a higher-priority process arrives). Running → Blocked happens when the process issues a request and must **wait for an event** (typically I/O). Blocked → Ready happens once that **event occurs** (e.g., I/O completes). Running → Exit happens on **completion**.

Q5. Why does the 7-state model add suspended states, and when would the OS actually use them?

Answer: It adds Ready-Suspend and Blocked-Suspend to represent a process whose memory image has been swapped out to secondary storage rather than kept in RAM. The OS uses these under real memory pressure — when there isn't enough RAM to hold every ready/blocked process, it suspends (swaps out) some of them to free space, and later activates (swaps back in) a suspended process once memory becomes available or its awaited event completes.

Q6. What does `fork()` return in the parent vs. the child, and why does that matter?

Answer: `fork()` returns the child's PID in the parent, and `0` in the child (and a negative value on failure). This matters because it's the *only* way the same code, now running in two separate processes, can tell which one it is and branch accordingly (e.g., `if (fork() == 0) { /* child code */ } else { /* parent code */ }`).

Q7. How many total processes exist after `n` sequential `fork()` calls?

Answer: 2^n — every existing process (the original plus every child created so far) calls `fork()` again at each step, so the count doubles each time.

Q8. What's the difference between a context switch and a mode switch?

Answer: A mode switch just flips the CPU between user mode and kernel mode (e.g., during a system call) without necessarily changing which process is logically executing. A context switch actually changes *which process* runs on the CPU — saving the current process's state to its PCB and loading another's — and inherently involves at least one mode switch as part of that work. So context switch time = mode switch time + additional overhead (saving/restoring registers, updating the PCB, etc.).

Q9. Why is thread creation faster than process creation?

Answer: Creating a process requires the OS to set up a brand-new address space, page tables, and copies of resources. Threads within the same process share the existing address space, code, data, and open files — creating a thread only means allocating a new stack and register set, not a whole new address space. This makes thread creation roughly ~100x faster.

Q10. What's the core weakness of user-level threads, and which threading model exists specifically to fix it?

Answer: The core weakness is the blocking problem — since the kernel only sees one process (not its individual user-level threads), if any one thread makes a blocking system call, the entire process gets blocked, since the kernel has nothing else of that process to schedule. The One-to-One model fixes this directly (each thread has its own kernel thread, so one blocking doesn't affect the others); Many-to-Many also fixes it while keeping some of the cheap-creation benefit of user-level threads.

Q11. Why can't a many-to-one threading model use multiple CPU cores even if they're available?

Answer: Because all of a process's user-level threads are mapped onto a single kernel thread. The kernel is the entity that actually gets scheduled onto CPU cores, and since there's only one kernel thread representing the entire process, only one core can ever be running that process's code at any instant — no matter how many user-level threads exist logically.

Q12. Why does Linux use a one-to-one threading model (NPTL) rather than many-to-one?

Answer: Because modern hardware is multi-core, and general-purpose workloads benefit far more from true parallelism and immunity to the total-blocking problem than from the marginal creation-speed advantage of many-to-one. NPTL trades slightly higher per-thread kernel overhead for threads that can run genuinely in parallel and that don't stall each other when one makes a blocking call — the right trade-off for a general-purpose OS.

Q13. Scenario: A multi-threaded process has one thread blocked on a slow disk read. Under many-to-one vs. one-to-one threading, what happens to the process's other threads?

Answer: Under **many-to-one**, all of the process's user-level threads share a single kernel thread. Since the kernel only sees that one kernel thread, and it's the one that issued the blocking read, the kernel blocks it — and because there's no other kernel thread for this process, *every* user-level thread of the process is effectively stalled too, even though most of them have nothing to do with the disk read. Under **one-to-one**, each user-level thread has its own kernel thread, so only the kernel thread that issued the blocking read is put in the blocked state; the kernel is free to schedule the process's *other* kernel threads (and hence their user-level threads) on the CPU as normal, completely unaffected by the blocked one.